LEADERS UNIVERSITY

MASTER · STATISTIQUE APPLIQUÉE

# Un café avec R

## Une mission à la fois

Du premier calcul à votre petit rapport réutilisable.

Un seul scénario · des maths simples · votre rythme

8 missions | R de base | Des résultats à montrer



INTRO · UN OUTIL, UNE HISTOIRE

# Pourquoi apprendre R avec ce café ?

R est un langage et un environnement pour calculer, analyser des données et dessiner des graphiques. Robert Gentleman et Ross Ihaka l'ont initialement créé à l'université d'Auckland, en s'inspirant du langage S. Une équipe et une communauté internationales le font évoluer.

### R exécute

Le moteur qui lit vos données et suit vos instructions. Gratuit et libre.

### RStudio organise

Un éditeur facultatif pour écrire, lancer et retrouver vos scripts R.

Pourquoi l'utiliser ici ? Votre script conserve une méthode que vous pouvez relancer sur un nouveau carnet, en gardant les mêmes règles. R est aussi utilisé dans des équipes en entreprise ; deux exemples documentés par le R Consortium :

### Pfizer

Un centre d'excellence R accompagne les usages de R et la reproductibilité.

### Swiss Re

Des équipes adoptent R et des outils ouverts pour leurs analyses.

VOTRE POINT DE DÉPART

Nous apprenons le même geste de base : lire un fichier, suivre une règle, vérifier et partager un résultat. Sources et crédits dans 05_sources.pdf ; ces marques ne sont pas des partenaires de la formation.

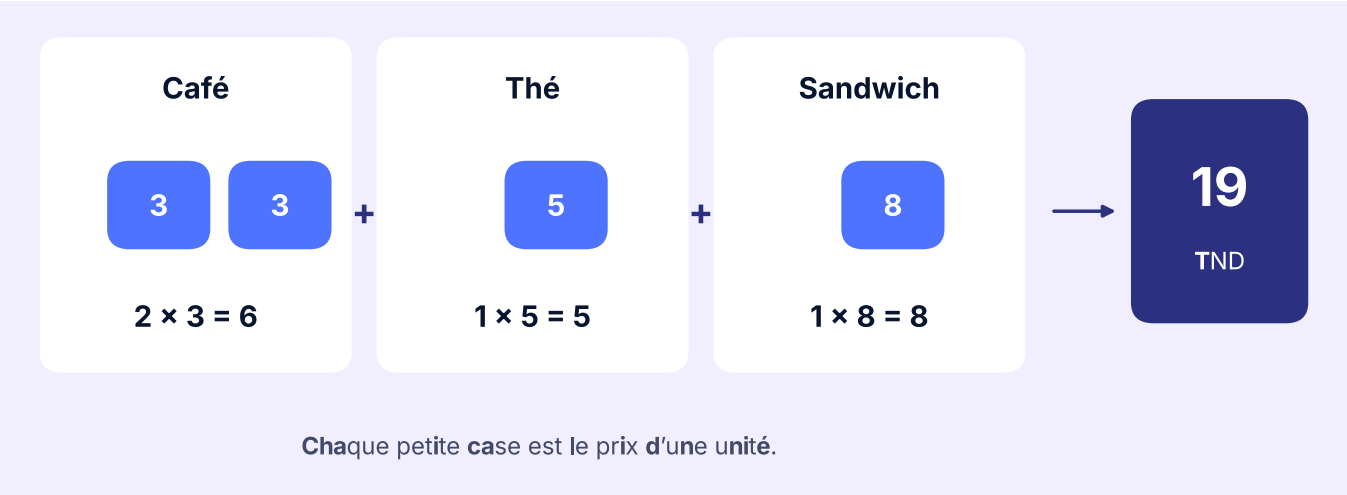
LE MONDE DE NORIA

# La Pause Noria : votre terrain de jeu

La Pause Noria est un café fictif. Sa gérante veut comprendre ses ventes, repérer les erreurs de saisie et choisir une action simple. Votre rôle : faire grandir le même carnet, du premier calcul à un outil réutilisable.

| Produit | Prix de vente | Coût des ingrédients |
|---|---|---|
| Café | 3 TND / unité | 1 TND / unité |
| Thé | 5 TND / unité | 2 TND / unité |
| Sandwich | 8 TND / unité | 3 TND / unité |

TND signifie dinar tunisien. Une quantité compte des unités vendues. Le chiffre d'affaires est l'argent des ventes. Le coût étudié ici est celui des ingrédients. La marge du cours est le chiffre d'affaires moins ce coût.




Dans votre tête · Un prix pour chaque unité, puis une addition : le panier vaut 19 TND.

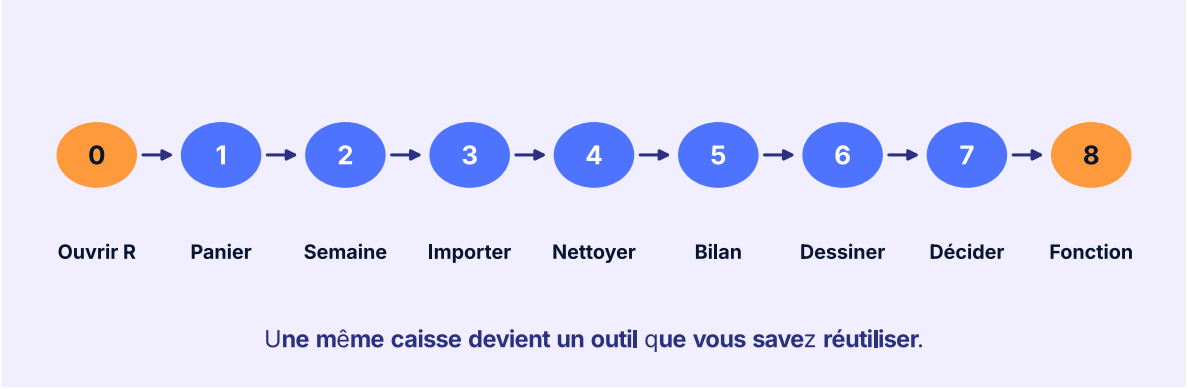
> MARGE NE VEUT PAS DIRE BÉNÉFICE
>
> Nous ne retirons ni loyer, ni salaires, ni électricité, ni autres charges. Une marge de 12 TND dans ce cours ne signifie donc pas un bénéfice de 12 TND pour une entreprise réelle. Les taxes ne sont pas déduites non plus.

Les données sont fabriquées pour apprendre : elles ne décrivent aucune vraie entreprise ni aucun vrai client. Elles ne contiennent pas de données personnelles. Gardez les fichiers bruts pour pouvoir expliquer chaque correction.

VOTRE CARTE DE PROGRESSION

# Une seule histoire, huit missions

Lire, essayer, enregistrer, vérifier, puis expliquer le résultat avec vos mots. Les pages se suivent : aucun bouton PDF n'est nécessaire. Faites une pause où vous le souhaitez et reprenez depuis votre script sauvegardé.




Dans votre tête · La même caisse grandit, une mission après l'autre.


| Mission | Vous allez savoir… | Page |
|---|---|---|
| 0 · Ouvrir R | Installer, écrire, sauver | 5 |
| 1 · Le panier | Créer des variables et calculer | 12 |
| 2 · Quatre jours | Organiser un vecteur, total et moyenne | 16 |
| 3 · Le carnet | Importer et lire un tableau | 19 |
| 4 · Les erreurs | Isoler un doublon et une absence | 23 |
| 5 · Le bilan | Créer des colonnes et regrouper | 26 |
| 6 · Les graphiques | Dessiner et enregistrer des barres | 31 |
| 7 · La décision | Calculer une part et partager | 35 |
| 8 · Le prochain fichier | Écrire une fonction réutilisable | 39 |

DÉPART · INSTALLER ET OUVRIR LE COURS

# Choisir votre porte d'entrée

R réalise les calculs. Un éditeur conserve vos instructions dans un fichier .R appelé script. RStudio réunit cet éditeur et une Console R. R de base 4.1 ou plus récent suffit ici : aucun paquet supplémentaire à installer.

| Votre ordinateur | Votre route dans ce guide |
|---|---|
| Windows · premier cours de R | Windows : R puis RStudio. La route la plus simple pour commencer. Page 6. |
| Windows · vous voulez Linux | WSL2 + Ubuntu : une option complète, si le Terminal vous est déjà familier. Page 7. |
| Ubuntu déjà installé | Ubuntu : installer R et travailler depuis le Terminal. Page 8. |
| Mac Intel ou puce Apple | macOS : choisir le bon R, puis RStudio si compatible. Page 9. |

UNE SEULE ROUTE SUFFIT

Choisir votre environnement et lire sa page. RStudio Windows utilise R Windows. La route WSL utilise R Linux dans Ubuntu : ses logiciels et ses chemins sont distincts. Ne pas mélanger les deux installations.

Pour Windows et Mac, ouvrir le dépôt ci-dessous dans votre navigateur. Choisir Code → Download ZIP. Extraire tout le ZIP ; sous Windows, clic droit → Extraire tout. Dans le dossier extrait, vous devez voir course.Rproj, data, missions et 00_verifier_installation.R. Garder ces fichiers ensemble. Git et un compte GitHub ne sont pas nécessaires.

https://github.com/jodouma/leaders-r-statistiques-18h

CONSOLE OU TERMINAL ?

Dans la Console R, le signe > précède les calculs : ne pas recopier ce signe. PowerShell et le Terminal Ubuntu servent à installer ou ouvrir les logiciels. Chaque bloc indique où saisir ses commandes. Quand R est prêt, passer à votre premier script après les pages d'installation.

DÉPART · INSTALLER ET OUVRIR LE COURS

# Windows : R, puis RStudio

1 · Installer R. Ouvrir l'adresse CRAN ci-dessous. Cliquer sur Download R for Windows, puis base et Download R… for Windows. Ouvrir le fichier .exe téléchargé et suivre les réglages proposés.

https://cran.r-project.org/

2 · Installer RStudio Desktop, édition gratuite Open Source, depuis la page Posit ci-dessous. Choisir le .exe Windows compatible avec votre version de Windows et l'ouvrir. Installer R avant RStudio. La page actuelle propose Windows 11 ; si votre système n'est pas compatible, utiliser R seul, ci-dessous.

https://docs.posit.co/ide/user/download.html

3 · Ouvrir le cours. Extraire le ZIP comme indiqué à la page « Choisir votre porte d'entrée ». Dans RStudio : File → Open Project → choisir course.Rproj dans le dossier extrait. La Console affiche la version de R. Cette route utilise R Windows, même si Ubuntu/WSL est aussi installé.

> SI VOUS UTILISEZ R SEUL
>
> Lancer R depuis le menu Démarrer. Copier l'adresse complète du dossier extrait depuis la barre d'adresse de l'Explorateur. Dans la Console R, remplacer CHEMIN_DU_DOSSIER ci-dessous par cette adresse, garder les guillemets et remplacer chaque \ par /. Exemple de forme : C:/Users/VOTRE_NOM/Documents/noria ; VOTRE_NOM est à remplacer.

```
setwd("CHEMIN_DU_DOSSIER")
```

4 · Vérifier, dans la Console R de RStudio ou de R seul :

```
getwd()
file.exists("data/orders_raw.csv")
source("00_verifier_installation.R")
```

getwd() doit afficher le dossier contenant course.Rproj. La deuxième ligne doit répondre TRUE ; la dernière doit afficher OK. Si FALSE, rouvrir le projet ou corriger setwd(). Vous pouvez ensuite passer à votre premier script. Avec R seul : File → New script ouvre un éditeur. La page du premier script indique quoi écrire, où enregistrer le .R et comment le relancer.

DÉPART · INSTALLER ET OUVRIR LE COURS

# Windows : la route Ubuntu / WSL2

Option pour utiliser Linux sous Windows 11 ou Windows 10 version 2004+ (build 19041+). Pour débuter avec R sans Linux, choisir la route Windows. Ici, tout R s'exécute dans Ubuntu ; RStudio Windows ne pilote pas ce R Linux.

1 · Menu Démarrer → chercher PowerShell → clic droit → Exécuter en tant qu'administrateur. Saisir dans PowerShell :

```
wsl --install -d Ubuntu
```

Redémarrer si demandé. Ouvrir Ubuntu depuis Démarrer ; attendre la fin de l'installation. Créer le nom et le mot de passe Linux demandés. Le mot de passe reste invisible pendant la saisie : c'est normal. Cette installation crée WSL2 par défaut.

2 · Dans la fenêtre Ubuntu, saisir une ligne à la fois. sudo demande le mot de passe Linux. git copie le cours public dans ~/cours/noria (~ désigne votre dossier personnel Linux).

```
sudo apt-get update
sudo apt-get install r-base git nano
mkdir -p ~/cours
cd ~/cours
git clone https://github.com/jodouma/leaders-r-statistiques-18h.git noria
cd noria
R
```

3 · Le signe > apparaît : vous êtes dans R. Saisir ces trois lignes :

```
getwd()
file.exists("data/orders_raw.csv")
source("00_verifier_installation.R")
```

Le dossier affiché finit par /cours/noria, puis vous lisez TRUE et OK. Pour modifier un script : quitter R avec q(save = "no"), puis saisir nano missions/01_panier.R dans Ubuntu. Écrire, sauver avec Ctrl+O puis Entrée, quitter avec Ctrl+X. Relancer R, puis source("missions/01_panier.R"). q(save = "no") quitte sans sauver les objets en mémoire ; vos fichiers .R enregistrés restent sur disque.

DÉPART · INSTALLER ET OUVRIR LE COURS

# Ubuntu : installer et ouvrir R

Cette route concerne un ordinateur déjà sous Ubuntu. Les versions LTS récentes (22.04 ou plus récent) fournissent un R adapté au cours. RStudio est facultatif : le Terminal et un petit éditeur suffisent.

1 · Ouvrir le Terminal avec Ctrl+Alt+T. Les lignes suivantes installent R, Git et l'éditeur nano. Saisir une ligne à la fois ; sudo demande votre mot de passe, sans afficher les caractères.

```
sudo apt-get update
sudo apt-get install r-base git nano
```

2 · Télécharger le cours dans votre dossier personnel (~) :

```
mkdir -p ~/cours
cd ~/cours
git clone https://github.com/jodouma/leaders-r-statistiques-18h.git noria
cd noria
R
```

Si vous avez déjà extrait le ZIP : dans Fichiers, ouvrir son dossier, appuyer sur Ctrl+L et copier l'adresse. À la place de l'étape 2, saisir les lignes ci-dessous en remplaçant le chemin fictif par cette adresse :

```
cd "/CHEMIN/DU/DOSSIER/EXTRAIT"
R
```

3 · Le signe > indique la Console R. Vérifier ici :

```
getwd()
file.exists("data/orders_raw.csv")
source("00_verifier_installation.R")
```

Le dossier doit contenir course.Rproj ; vous devez lire TRUE puis OK. Pour écrire : dans R, q(save = "no") revient au Terminal sans effacer vos .R enregistrés. Taper nano missions/01_panier.R ; modifier, Ctrl+O puis Entrée pour sauver, Ctrl+X pour quitter. Relancer R et source("missions/01_panier.R").

DÉPART · INSTALLER ET OUVRIR LE COURS

# Mac : Intel ou puce Apple

1 · Identifier votre Mac : menu Apple → À propos de ce Mac. Repérer la version de macOS, puis Puce Apple (M1, M2, etc.) ou Processeur Intel.

2 · Installer R depuis CRAN, à l'adresse ci-dessous : choisir Download R for macOS. Pour une puce Apple, prendre le .pkg arm64 compatible ; pour Intel, prendre le .pkg x86_64 compatible. Ouvrir ce fichier .pkg et suivre l'installation. Lire les versions de macOS prises en charge avant de télécharger.

https://cran.r-project.org/

3 · Installer RStudio Desktop gratuit depuis la page Posit ci-dessous. Choisir le .dmg compatible, l'ouvrir, puis glisser RStudio dans Applications. Le téléchargement actuel est Universal (Intel et puce Apple), pour macOS 15+. Si votre système n'est pas compatible, suivre la route R seul ci-dessous.

https://docs.posit.co/ide/user/download.html

4 · Extraire le ZIP du cours (double-clic dans Finder). Dans RStudio : File → Open Project → choisir course.Rproj dans ce dossier. RStudio utilise alors R installé sur ce Mac.

> SI VOUS UTILISEZ R SEUL
>
> Ouvrir R.app depuis Applications. Dans Finder, clic droit sur le dossier extrait ; maintenir Option et choisir Copier comme nom de chemin. Dans la Console R, remplacer CHEMIN_DU_DOSSIER par ce chemin, entre guillemets. Pour modifier un script existant : TextEdit → Fichier → Ouvrir → choisir son fichier .R ; conserver le texte brut et sauver avec Cmd+S. La page du premier script indique quoi écrire, où enregistrer et comment relancer avec source().

```
setwd("CHEMIN_DU_DOSSIER")
```

5 · Dans la Console R de RStudio ou de R seul, vérifier :

```
getwd()
file.exists("data/orders_raw.csv")
source("00_verifier_installation.R")
```

getwd() affiche le dossier contenant course.Rproj ; file.exists() répond TRUE ; la dernière ligne affiche OK. Si FALSE, rouvrir le projet ou corriger setwd(). Passer ensuite à votre premier script. Aucun Homebrew ni compilateur n'est nécessaire pour les missions.

DÉPART · VOTRE PREMIER FICHIER

# Écrire et enregistrer votre premier script

Créer examples/00_depart.R dans le dossier du cours. Choisir votre éditeur ci-dessous ; écrire les deux lignes du bloc sombre, puis enregistrer le fichier avec son extension .R.

| Votre outil | Créer, écrire, sauvegarder |
|---|---|
| RStudio | File → New File → R Script ; Ctrl+S ou Cmd+S ; choisir examples/00_depart.R. |
| R Windows seul | File → New script ; File → Save as ; choisir examples/00_depart.R. |
| R Mac seul | Dans TextEdit : nouveau document, Format → Convertir au format Texte ; enregistrer examples/00_depart.R sans ajouter .txt. |
| Ubuntu / WSL | Quitter R avec q(save = "no"). Dans le Terminal : nano examples/00_depart.R. Ctrl+O, Entrée, puis Ctrl+X ; relancer R. |

```
# Mon premier script Noria
print(2 + 2)
```

# commence un commentaire, ignoré par R. print() affiche une réponse, même lorsqu'un script entier est lancé. Dans la Console R, lancer le fichier enregistré :

```
source("examples/00_depart.R")
```

DÉPART · VOTRE PREMIER FICHIER

# Écrire et enregistrer votre premier script

UNE PREMIÈRE RÉUSSITE DURABLE

Vous lisez 4. Fermer puis rouvrir votre fichier : les deux lignes sont toujours là. Relancer le fichier depuis la Console avec source(). Les objets temporaires peuvent disparaître ; le script enregistré permet de les recréer.

## Reconnaître la Console et retrouver les fichiers

La Console R attend une commande après >. Un + indique souvent une parenthèse ou un guillemet à fermer : Échap dans RStudio, Ctrl+C dans le Terminal, annule la saisie en cours. Dans RStudio, Ctrl+Entrée ou Cmd+Entrée exécute la ligne ou la sélection ; Source exécute tout le fichier.

```
print(getwd())
print(file.exists("data/orders_raw.csv"))
source("00_verifier_installation.R")
```

getwd() affiche le dossier où R cherche. file.exists() répond TRUE si le chemin existe, FALSE sinon. Les / séparent les sous-dossiers. Ne pas recopier > ni les résultats dans le script : seuls les blocs de commandes vont dans le fichier.

AVANT LA MISSION 1

Vous obtenez 4 et TRUE ; la préparation affiche OK. Si FALSE, rouvrir course.Rproj dans RStudio ou revenir au dossier indiqué par votre route d'installation. Garder data, missions et examples ensemble dans le même dossier.

MISSION 1 · LA PAUSE NORIA

# Le tout premier ticket

Objectif · Calculer le chiffre d'affaires et la marge d'un panier de 2 cafés, 1 thé et 1 sandwich.

Pourquoi ? Avant d'analyser beaucoup de lignes, nous voulons comprendre ce que signifie chaque nombre. Nous faisons une fois le calcul à la main pour reconnaître ensuite une réponse plausible de R.

| Étape à la main | Calcul | Résultat |
|---|---|---|
| Ventes | 2 × 3 + 1 × 5 + 1 × 8 | 19 TND |
| Ingrédients | 2 × 1 + 1 × 2 + 1 × 3 | 7 TND |
| Marge | 19 − 7 | 12 TND |

VOTRE PRÉDICTION

Si nous vendons un café de plus, les ventes augmentent de 3 TND et la marge de 2 TND. Gardez cette prédiction : elle servira à tester votre script.

## Donner un nom à un résultat

Dans R, * multiplie et / divise. + additionne et - soustrait. <- donne une valeur à un nom : on parle d'objet. Dans la Console, écrire son nom affiche sa valeur ; dans un script lancé avec source(), utiliser print(nom). Un nom sans espace est plus facile à réutiliser.




Dans votre tête · Un nom à gauche conserve une valeur à droite.

MISSION 1 · LA PAUSE NORIA

# Donner un nom à un résultat

```
chiffre_affaires <- 2 * 3 + 5 + 8
cout_total <- 2 * 1 + 2 + 3
marge <- chiffre_affaires - cout_total
print(chiffre_affaires)
print(cout_total)
print(marge)
```

| Nom affiché | Valeur attendue | Unité |
|---|---|---|
| chiffre_affaires | 19 | TND |
| cout_total | 7 | TND |
| marge | 12 | TND |

UNE ERREUR, UNE RÉPARATION

object not found signifie objet introuvable. R distingue marge de Marge. Corriger le nom, puis exécuter les lignes qui créent les objets avant celles qui les utilisent.

## À vous : le premier panier

Ouvrir missions/01_panier.R. Le fichier conserve les consignes et votre réponse. Enregistrer avec le même nom. Les premières quantités du panier sont 2 cafés, 1 thé et 1 sandwich.

Créer les objets prix_cafe, quantite_cafe, prix_the, quantite_the, prix_sandwich et quantite_sandwich. Leurs valeurs sont 3, 2, 5, 1, 8 et 1. Créer ensuite total_panier, cout_panier et marge_panier.

MISSION 1 · LA PAUSE NORIA

# À vous : le premier panier

```
total_panier <- prix_cafe * quantite_cafe +
  prix_the * quantite_the +
  prix_sandwich * quantite_sandwich
cout_panier <- quantite_cafe * 1 +
  quantite_the * 2 + quantite_sandwich * 3
marge_panier <- total_panier - cout_panier
print(total_panier)
print(cout_panier)
print(marge_panier)
```

VOTRE ESSAI ANALOGUE

Sans toucher aux objets de la preuve, calculer dans un autre objet le panier avec 3 cafés. Prédire le résultat avant de lancer. Vous devez obtenir 22 TND de ventes et 14 TND de marge.

Exécuter tout le script. Les valeurs de preuve restent celles du panier initial : 19, 7 et 12 TND.

## Première porte ouverte : calculer

| Preuve de mission 1 | Attendu |
|---|---|
| total_panier | 19 TND |
| cout_panier | 7 TND |
| marge_panier | 12 TND |

MISSION 1 · LA PAUSE NORIA

# Première porte ouverte : calculer




Dans votre tête · Le ticket de 19 TND contient 7 TND de coût et 12 TND de marge.

EN RÉSUMÉ · VOUS SAVEZ MAINTENANT

Créer un objet avec <-, effectuer un calcul, afficher un résultat et vérifier son unité. Vous savez qu'un café supplémentaire apporte 3 TND de ventes et 2 TND de marge.

Preuve : missions/01_panier.R enregistré et ses résultats affichés dans la console. Il n'y a pas de fichier de sortie à fabriquer pour cette mission. Pour vérifier, source() exécute un autre fichier R : Le dossier checks contient les contrôles : exécuter la commande indiquée et garder leurs fichiers fermés pendant l'exercice.

```
source("checks/01_panier.R")
```

MISSION 2 · LA PAUSE NORIA

# Quatre jours dans la même boîte

Objectif · Ranger quatre chiffres d'affaires journaliers dans un seul objet, puis calculer leur total et leur moyenne.

Pourquoi ? Copier une addition de plus en plus longue devient fragile. Un vecteur rassemble plusieurs valeurs de même nature. Ici, une valeur correspond à un jour.

| Jour | Chiffre d'affaires |
|---|---|
| Lundi | 19 TND |
| Mardi | 27 TND |
| Mercredi | 33 TND |
| Jeudi | 21 TND |

À la main : 19 + 27 + 33 + 21 = 100 TND. La moyenne est le total divisé par le nombre de jours : 100 / 4 = 25 TND par jour. Cela ne signifie pas que chaque jour a vendu 25 TND.




La moyenne partage le total en quatre parts égales.

Dans votre tête · Imaginer quatre parts égales de 25 TND, même si les jours sont différents.

## Un vecteur, quatre valeurs

c() combine plusieurs valeurs en un vecteur ; les virgules séparent les valeurs. Une fonction est une commande qui reçoit une information entre parenthèses et renvoie un résultat. sum() fait le total, length() compte les éléments et mean() calcule leur moyenne.

MISSION 2 · LA PAUSE NORIA

# Un vecteur, quatre valeurs




Dans votre tête · Une case par jour. Son numéro permet de retrouver une valeur.

```
ventes_journalieres <- c(19, 27, 33, 21)
print(sum(ventes_journalieres))
print(length(ventes_journalieres))
print(mean(ventes_journalieres))
print(ventes_journalieres[2])
```

| Commande | Réponse attendue |
|---|---|
| sum(ventes_journalieres) | 100 TND sur les quatre jours |
| length(ventes_journalieres) | 4 jours |
| mean(ventes_journalieres) | 25 TND par jour |
| ventes_journalieres[2] | 27 TND : deuxième valeur |

VIRGULE OU POINT ?

R attend un point pour les décimales : 3.5. Dans c(19, 27), la virgule sépare deux nombres. Dans R, la première position porte le numéro 1.

MISSION 2 · LA PAUSE NORIA

# Un vendredi prometteur

Ouvrir missions/02_semaine.R. Créer ventes_journalieres avec les quatre jours 19, 27, 33 et 21 TND. Garder ce vecteur pour la preuve. Malgré le mot semaine du fichier, seules quatre journées sont observées.

- Créer total_semaine avec sum() et moyenne_jour avec mean(). Afficher ces deux objets. print(total_semaine) et print(moyenne_jour) affichent aussi les résultats pendant Source.
- Compter les jours avec length(). Vérifier la moyenne par une division.
- Dans un nouvel objet ventes_avec_vendredi, ajouter un cinquième jour de 30 TND. Prévoir le nouveau total, puis contrôler avec R.
- Enregistrer et relancer le script depuis son début. Les objets de preuve total_semaine et moyenne_jour restent calculés sur les quatre jours.

> VIRGULES ET GUILLEMETS
>
> Une valeur entre guillemets comme "30" est du texte. Ici écrire 30 sans guillemets. Garder les parenthèses de c() et séparer les valeurs par des virgules.

## La série est sous contrôle

| Preuve sur quatre jours | Attendu |
|---|---|
| ventes_journalieres | 19, 27, 33, 21 TND |
| total_semaine | 100 TND |
| moyenne_jour | 25 TND par jour |

Contrôle de votre essai à cinq jours : 130 TND au total, 5 jours, moyenne 26 TND par jour. Cet essai aide à comprendre ; la preuve demandée reste le vecteur de quatre jours.

> EN RÉSUMÉ · VOUS SAVEZ MAINTENANT
>
> Créer un vecteur, compter ses éléments, en extraire un et calculer total et moyenne. Vous savez vérifier la moyenne avec une division simple.

```
source("checks/02_semaine.R")
```

MISSION 3 · LA PAUSE NORIA

# La caisse devient un tableau

Objectif · Importer le fichier de caisse et lire ses lignes et ses colonnes sans les modifier.

Pourquoi ? Des valeurs isolées ne disent pas quel produit a été vendu ni à quelle date. Un tableau conserve ces informations ensemble. En R, ce type de tableau se nomme data frame.

| id | produit | quantite |
|---|---|---|
| O001 | Cafe | 2 |
| O002 | The | 1 |
| O003 | Sandwich | 1 |

À la main : la deuxième ligne décrit 1 thé. La colonne quantite contient 2, 1, 1 dans cet extrait. Une ligne décrit ici une vente d'un produit ; elle ne décrit pas forcément un client ni une journée entière. Pour simplifier, chaque ligne est une commande d'un seul type de produit ; plusieurs unités sont possibles. Le nombre de lignes n'est pas le nombre de clients.

> LE BRUT A DES SURPRISES
>
> Le fichier brut contient 14 lignes. Certaines demandent une vérification : les compter ne prouve pas encore que les 14 ventes sont utilisables. Nous traiterons ces surprises en mission 4.

## Les mots du fichier

| Colonne R | Signification | Exemple / unité |
|---|---|---|
| id | Identifiant de vente | O001 |
| date | Jour de vente | 2026-10-05 |
| produit | Nom du produit | Cafe, The, Sandwich |
| quantite | Nombre d'unités vendues | 2 unités |
| prix_unitaire | Prix de vente d'une unité | 3 TND / unité |
| cout_unitaire | Coût des ingrédients d'une unité | 1 TND / unité |

MISSION 3 · LA PAUSE NORIA

# Les mots du fichier

1 **ligne** = 1 **vente enregistrée** 1 **colonne** = 1 **information**

| id | date | produit | quantite | prix unitaire | coût unitaire |
|---|---|---|---|---|---|
| O001 | 2026-10-05 | Cafe | 2 | 3 | 1 |
| O002 | 2026-10-05 | The | 1 | 5 | 2 |
| O003 | 2026-10-05 | Sandwich | 1 | 8 | 3 |

**La cellule orange : quantité du thé = 1.**

Dans votre tête · Lire une cellule, puis toute sa ligne ou toute sa colonne.

Les noms évitent accents et espaces pour être faciles à saisir. Cafe correspond à café, The à thé. Une date écrite 2026-10-05 se lit année-mois-jour : 5 octobre 2026.

> LIRE L'UNITÉ ÉVITE UNE ERREUR
>
> prix_unitaire est le prix d'une seule unité. Pour 2 cafés, le chiffre d'affaires de la ligne sera 2 × 3 = 6 TND. Additionner seulement les prix unitaires donnerait une réponse à une autre question.

## Importer, puis regarder

Un CSV est un fichier texte qui range un tableau : une ligne d'en-têtes puis les données. read.csv() lit ce fichier ; le chemin va entre guillemets. <- garde le tableau dans commandes. head() affiche le début ; nrow() compte les lignes et names() donne les noms des colonnes.

MISSION 3 · LA PAUSE NORIA

# Importer, puis regarder

```
commandes <- read.csv("data/orders_raw.csv")
print(head(commandes))
print(nrow(commandes))
print(names(commandes))
```

| Ce que vous vérifiez | Attendu |
|---|---|
| nrow(commandes) | 14 lignes |
| names(commandes) | Les six noms du dictionnaire |
| head(commandes) | Les premières ventes, avec leurs en-têtes |

> CANNOT OPEN FILE
>
> R ne trouve pas le fichier. Vérifier getwd(), le nom orders_raw.csv, puis le sous-dossier data. Utiliser le chemin du dossier extrait sur votre propre ordinateur.

## Retrouver une vente précise

Le signe $ choisit une colonne : commandes$quantite désigne toutes les quantités. Les crochets [ligne, colonne] choisissent une case. Laisser la colonne vide choisit la ligne entière.

```
premiere_commande <- commandes[1, ]
print(commandes[2, "produit"])
print(commandes$quantite)
```

MISSION 3 · LA PAUSE NORIA

# Retrouver une vente précise

unique() conserve chaque valeur distincte une seule fois. Sur la colonne produit, cette fonction permet de voir les produits disponibles sans répétition.

```
produits <- unique(commandes$produit)
print(produits)
```

Dans missions/03_importer.R, importer le brut, créer premiere_commande et produits, puis afficher les résultats. Pour un essai analogue, retrouver aussi le produit et la quantité de la troisième ligne. Expliquer pourquoi un prix unitaire ne suffit pas pour calculer les ventes.

## Vous savez lire la caisse

| Preuve de mission 3 | Attendu |
|---|---|
| commandes | 14 lignes, 6 colonnes |
| premiere_commande | O001 ; Cafe ; 2 unités |
| produits | Cafe, The et Sandwich, sans répétition |
| Votre essai sur la troisième ligne | Sandwich ; 1 unité ; prix unitaire 8 TND |

EN RÉSUMÉ · VOUS SAVEZ MAINTENANT

Importer un CSV, repérer lignes et colonnes, lire avec $ et [ ], et trouver des valeurs distinctes avec unique(). Vous savez donner le sens et l'unité d'une valeur.

```
source("checks/03_importer.R")
```

MISSION 4 · LA PAUSE NORIA

# Deux surprises dans la caisse

Objectif · Retirer une copie exacte et mettre de côté une vente dont la quantité manque.

Pourquoi ? Compter deux fois la même vente gonfle les ventes. Inventer une quantité transforme une inconnue en fausse certitude. Nous allons garder une trace de ce que nous changeons.

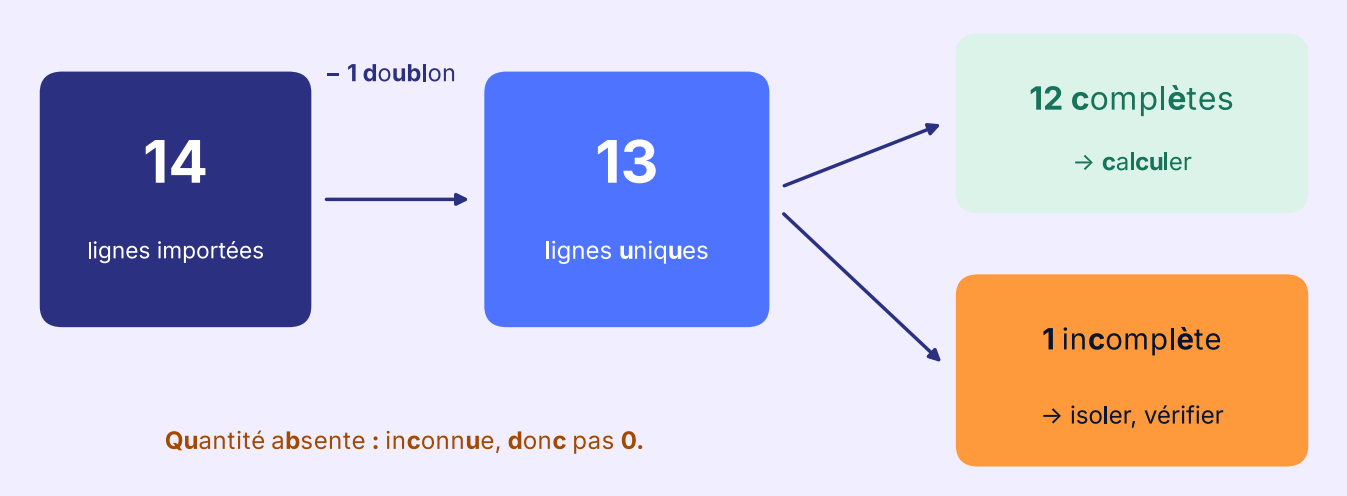
| Mini-exemple à la main | Traitement | Lignes retenues |
|---|---|---|
| A : 2 cafés ; A : 2 cafés | La deuxième ligne est une copie exacte | Une ligne A |
| B : quantité inconnue | Mettre B à part pour demander la quantité | Aucune ligne B pour les calculs |

CE QUI MANQUE RESTE INCONNU

Dans R, NA signifie valeur manquante. NA n'est ni zéro ni le texte "NA" à inventer. Ici, nous excluons provisoirement la ligne des calculs et conservons un fichier des lignes à vérifier.

## Une règle visible pour chaque ligne

Sur un tableau entier, unique() garde seulement une copie de chaque ligne entièrement identique. is.na() pose la question « est-ce manquant ? » et renvoie TRUE ou FALSE pour chaque valeur. ! inverse ces réponses.




Dans votre tête · On conserve 12 lignes et une ligne à compléter. Une absence n'est pas zéro.

MISSION 4 · LA PAUSE NORIA

# Une règle visible pour chaque ligne

```
sans_doublon <- unique(commandes)
print(is.na(sans_doublon$quantite))
print(!is.na(sans_doublon$quantite))
```

Dans [règle, ], R garde les lignes pour lesquelles la règle est TRUE. Avant la virgule : les lignes. Après la virgule vide : toutes les colonnes. Combinons maintenant ces gestes connus.

```
a_completer <- sans_doublon[
  is.na(sans_doublon$quantite), ]
propres <- sans_doublon[
  !is.na(sans_doublon$quantite), ]
```

| Étape | Nombre de lignes |
|---|---|
| Brut → sans copie exacte | 14 → 13 |
| Quantité manquante, à compléter | 1 |
| Lignes conservées pour calculer | 12 |

## Conserver une caisse fiable

write.csv() écrit un tableau dans un fichier CSV. Son premier argument est le tableau, le second est le chemin de sortie. row.names = FALSE évite d'ajouter une colonne de numéros de lignes. FALSE veut dire faux.

```
write.csv(propres,
  "outputs/04_commandes_propres.csv",
  row.names = FALSE)
write.csv(a_completer,
  "outputs/04_commandes_a_completer.csv",
  row.names = FALSE)
```

MISSION 4 · LA PAUSE NORIA

# Conserver une caisse fiable

- Dans le script de mission 4, importer le brut puis appliquer les deux règles.
- Afficher les trois nombres 13, 1 et 12. Identifier la vente à vérifier : O013.
- Enregistrer les deux CSV dans outputs. Ouvrir le fichier à vérifier et s'assurer que la quantité n'a pas été remplacée par zéro.
- Écrire un commentaire : 14 lignes brutes = 1 copie retirée + 1 ligne mise à part + 12 lignes conservées.

SI COMMANDES MANQUE

Le script doit commencer par l'import read.csv(). Après un redémarrage, les objets de la session précédente peuvent avoir disparu : votre fichier doit pouvoir les recréer.

## Une preuve de nettoyage

| Fichier dans outputs | Ce qu'il contient |
|---|---|
| 04_commandes_propres.csv | 12 lignes utilisables, sans copie exacte |
| 04_commandes_a_completer.csv | O013, quantité manquante conservée |

EN RÉSUMÉ · VOUS SAVEZ MAINTENANT

Appliquer une règle simple de nettoyage, repérer NA, filtrer des lignes et exporter un tableau. Vous savez expliquer ce qui a été retiré et pourquoi.

Nous retirons ici seulement une copie exacte. Deux ventes différentes portant des produits et des prix identiques peuvent être toutes deux réelles. Une règle de nettoyage dépend toujours de la signification des données.

Enregistrer missions/04_nettoyer.R et vérifier depuis la console : source("checks/04_nettoyer.R"). Le contrôle relance votre script ; les fichiers demandés doivent aussi être lisibles dans outputs.

MISSION 5 · LA PAUSE NORIA

# Chaque ligne gagne trois colonnes

Objectif · Calculer ventes, coût et marge pour chaque vente, puis réunir les résultats par produit.

Pourquoi ? La gérante veut comparer les produits sur la période entière. Nous passons du prix d'une unité à l'argent correspondant aux quantités réellement vendues.

| Exemple : 2 cafés | Calcul à la main | Résultat |
|---|---|---|
| Chiffre d'affaires | 2 × 3 | 6 TND |
| Coût des ingrédients | 2 × 1 | 2 TND |
| Marge | 6 – 2 | 4 TND |

R applique une multiplication colonne par colonne, ligne par ligne : une quantité est multipliée par son propre prix unitaire. Les nouvelles colonnes auront autant de valeurs que le tableau a de lignes.

## Calculer sans recopier douze fois

Le début fourni dans missions/05_analyser.R prépare chemin_propres et importe le tableau. Son if signifie « si » : si votre export existe, il est choisi ; sinon le fichier de référence est annoncé. Vous pouvez utiliser cette préparation pour commencer directement les calculs. propres$nouveau_nom <- … crée une colonne.

| quantite | | prix_unitaire | | ca | marge |
|---|---|---|---|---|---|
| 2 | × | 3 | = | 6 | 4 |
| 1 | × | 5 | = | 5 | 3 |
| 1 | × | 8 | = | 8 | 5 |

R répète la même règle sur toutes les lignes.

Dans votre tête · La même règle se répète sur chaque ligne du tableau.

MISSION 5 · LA PAUSE NORIA

# Calculer sans recopier douze fois

```
propres <- read.csv(chemin_propres)
propres$chiffre_affaires <-
  propres$quantite * propres$prix_unitaire
propres$cout_total <-
  propres$quantite * propres$cout_unitaire
propres$marge <-
  propres$chiffre_affaires - propres$cout_total
print(sum(propres$chiffre_affaires))
print(sum(propres$cout_total))
print(sum(propres$marge))
```

| Sur les 12 ventes retenues | Attendu |
|---|---|
| Chiffre d'affaires total | 100 TND |
| Coût total des ingrédients | 37 TND |
| Marge totale | 63 TND |

UNE AIDE POUR REPRENDRE ICI

Le début fourni des missions 5, 6 et 7 peut choisir le fichier de référence si vous n'avez pas encore votre export de mission 4. Il permet de travailler sur ces missions séparément. Le message indique le fichier utilisé ; cette aide ne valide pas la mission 4.

## Réunir les ventes par produit

aggregate() réunit les lignes par groupe. Dans chiffre_affaires ~ produit, ~ se lit « selon » : total des ventes selon le produit. data désigne le tableau ; FUN = sum choisit le total. Ces mots sont des noms d'arguments.

MISSION 5 · LA PAUSE NORIA

# Réunir les ventes par produit

**Café**

**6 + 9 + 12 + 3**

↓

**CA : 30 TND**

Coût **10** · marge **20**

**Sandwich**

**8 + 8 + 16 + 8**

↓

**CA : 40 TND**

Coût 15 · marge 25

**Thé**

5 **+ 10 +** 5 **+ 10**

↓

**CA : 30 TND**

Coût **12** · marge **18**

Dans votre tête · Réunir les lignes du même produit, puis additionner leurs valeurs.

```
print(aggregate(chiffre_affaires ~ produit,
  data = propres, FUN = sum))
```

| Produit | Chiffre d'affaires |
|---|---|
| Cafe | 30 TND |
| Sandwich | 40 TND |
| The | 30 TND |

MISSION 5 · LA PAUSE NORIA

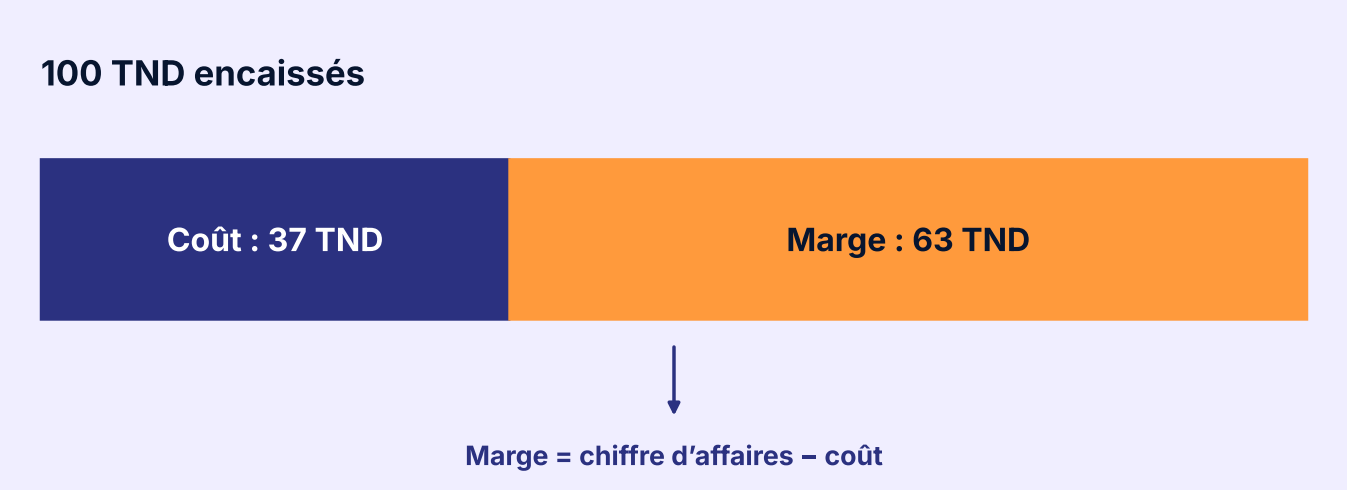
# Réunir les ventes par produit

VOTRE ESSAI ANALOGUE

Dans missions/05_analyser.R, adapter cette ligne pour totaliser la marge selon le produit. Vérifier 20 TND pour Cafe, 25 pour Sandwich et 18 pour The. La page suivante rassemble les trois mesures dans un seul tableau.

## Trois mesures, un seul résumé

cbind() place plusieurs colonnes côte à côte. Dans aggregate(), il permet de demander le même total pour chiffre_affaires, cout_total et marge. Le résultat sera nommé bilan_produits.




Dans votre tête · 100 TND de ventes = 37 TND de coût + 63 TND de marge sur ingrédients.

```
bilan_produits <- aggregate(
  cbind(chiffre_affaires, cout_total, marge) ~
    produit, data = propres, FUN = sum)
print(bilan_produits)
total_ca <- sum(propres$chiffre_affaires)
total_marge <- sum(propres$marge)
```

Pour un résumé par jour, reprendre la même recette et remplacer produit par date ; nommer le résultat bilan_jours. Aucun nouveau calcul n'est nécessaire : seul le groupe change.

MISSION 5 · LA PAUSE NORIA

# Trois mesures, un seul résumé

```
write.csv(bilan_produits,
  "outputs/05_bilan_produits.csv",
  row.names = FALSE)
```

VOTRE CONTRÔLE

Afficher total_ca = 100 TND et total_marge = 63 TND. Ouvrir le CSV enregistré. Vous devez y retrouver trois produits et les trois mesures, avec des en-têtes. Le bilan par jour doit retrouver 19, 27, 33 et 21 TND de ventes, dans l'ordre des dates.

## Le tableau de bord est prêt

| Produit | Ventes / TND | Marge / TND |
|---|---|---|
| Cafe | 30 | 20 |
| Sandwich | 40 | 25 |
| The | 30 | 18 |
| Total | 100 | 63 |

EN RÉSUMÉ · VOUS SAVEZ MAINTENANT

Créer une colonne à partir de deux autres, totaliser une colonne et calculer des totaux par groupe avec aggregate(). Vous savez vérifier qu'un résumé retrouve le total général.

La marge la plus élevée sur cette période est celle du sandwich : 25 TND. Cela ne mesure ni la satisfaction client, ni le temps de préparation, ni la rentabilité complète du café.

Enregistrer missions/05_analyser.R et vérifier depuis la console : source("checks/05_analyser.R"). Le contrôle relance votre script ; les fichiers demandés doivent aussi être lisibles dans outputs.

MISSION 6 · LA PAUSE NORIA

# Un regard suffit pour comparer

Objectif · Dessiner un diagramme en barres des ventes par produit, puis un diagramme des marges.

**30**

**TND**

Chiffre d'affaires du café

**10**

**cafés**

Quantité de café vendue

Même produit, deux questions : combien d'argent ? combien d'unités ?

Dans votre tête · 30 TND de ventes et 10 cafés répondent à deux questions différentes.

Pourquoi ? Un tableau donne les nombres précis. Un graphique permet de comparer rapidement leurs tailles. Une barre correspondra à un produit ; sa hauteur représentera une somme en TND.

| Notre brouillon à la main | Hauteur à représenter |
|---|---|
| Cafe | 30 TND de ventes |
| Sandwich | 40 TND de ventes |
| The | 30 TND de ventes |

Dessinez trois rectangles sur papier : deux de même hauteur et celui du sandwich plus haut. Placez zéro à la base. Ce brouillon vous permettra de reconnaître un graphique incohérent.

> LA QUESTION RESTE VISIBLE
>
> Titre : chiffre d'affaires par produit. Axe vertical : ventes en TND. Noms sous les barres : les produits. Sans ces repères, une belle image peut rester incompréhensible.

## R dessine vos barres

barplot() dessine des barres. height reçoit leurs hauteurs ; names.arg reçoit leurs noms. main indique le titre et ylab le nom de l'axe vertical. Les textes vont entre guillemets.

MISSION 6 · LA PAUSE NORIA

# R dessine vos barres

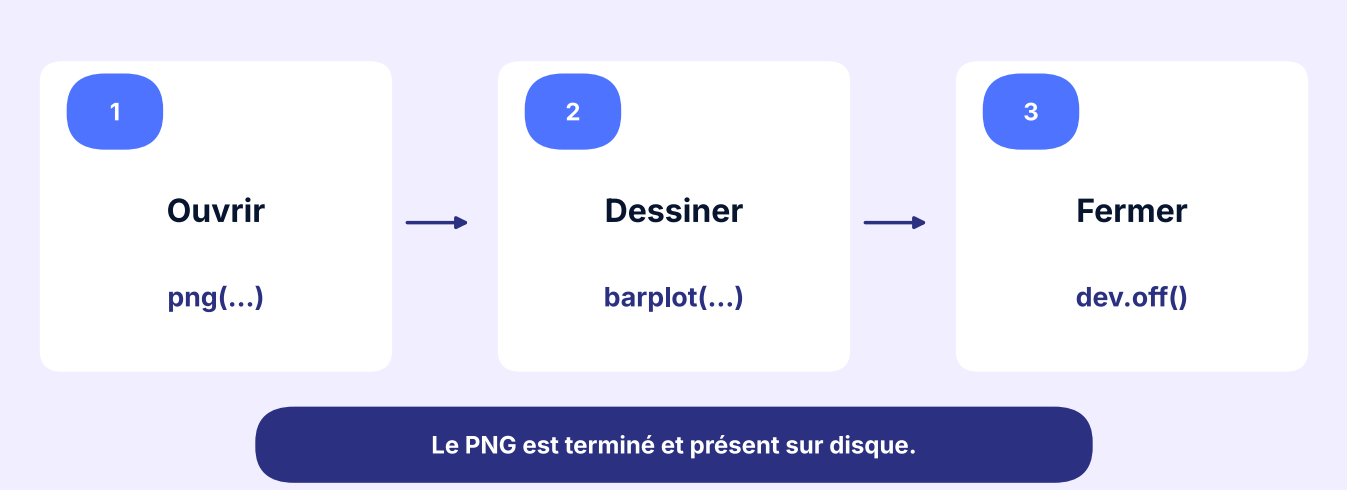
```
barplot(
  height = bilan_produits$chiffre_affaires,
  names.arg = bilan_produits$produit,
  main = "Ventes par produit",
  ylab = "Chiffre d affaires (TND)",
  col = "steelblue")
```

col choisit la couleur ; steelblue est le nom anglais d'un bleu. Dans RStudio, le résultat apparaît dans Plots. Dans le Terminal, une fenêtre peut ne pas s'ouvrir : utiliser alors png() → barplot() → dev.off() comme expliqué juste après, puis ouvrir le fichier enregistré.

> LECTURE ATTENDUE
>
> Deux barres de 30 TND encadrent une barre de 40 TND si les produits sont rangés Cafe, Sandwich, The. L'ordre peut changer : toujours lire les noms, jamais seulement la position.

## Une image qui reste après la séance

png() ouvre un fichier image : les graphiques suivants y sont écrits. dev.off() ferme ce fichier et termine son enregistrement. Il faut exécuter toute la séquence, y compris sa dernière ligne.




Dans votre tête · Ouvrir le fichier, dessiner, fermer pour enregistrer.

MISSION 6 · LA PAUSE NORIA

# Une image qui reste après la séance

```
png("outputs/06_ca_produits.png",
  width = 900, height = 600)
barplot(bilan_produits$chiffre_affaires,
  names.arg = bilan_produits$produit,
  main = "Ventes par produit",
  ylab = "Ventes (TND)", col = "steelblue")
dev.off()
```

RETROUVER LE PNG DANS UBUNTU / WSL

Après dev.off(), le fichier est dans outputs/. Sous WSL : quitter R avec q(save = "no"), puis saisir explorer.exe outputs dans le Terminal Ubuntu pour ouvrir ce dossier dans Windows. Sous Ubuntu, ouvrir outputs/ dans le gestionnaire de fichiers. Relancer R avant la suite.

- Exécuter missions/06_visualiser.R depuis son début. La préparation fournie recrée le bilan et peut utiliser la référence si nécessaire ; vous complétez ensuite la partie graphique.
- Créer et enregistrer le graphique des ventes. Adapter ensuite ce modèle au tableau bilan_produits.
- Nommer le deuxième fichier outputs/06_marge_produits.png. Adapter aussi le titre et le nom de l'axe.
- Ouvrir les deux images depuis outputs et comparer leurs barres aux tableaux.

IMAGE ABSENTE OU VIDE ?

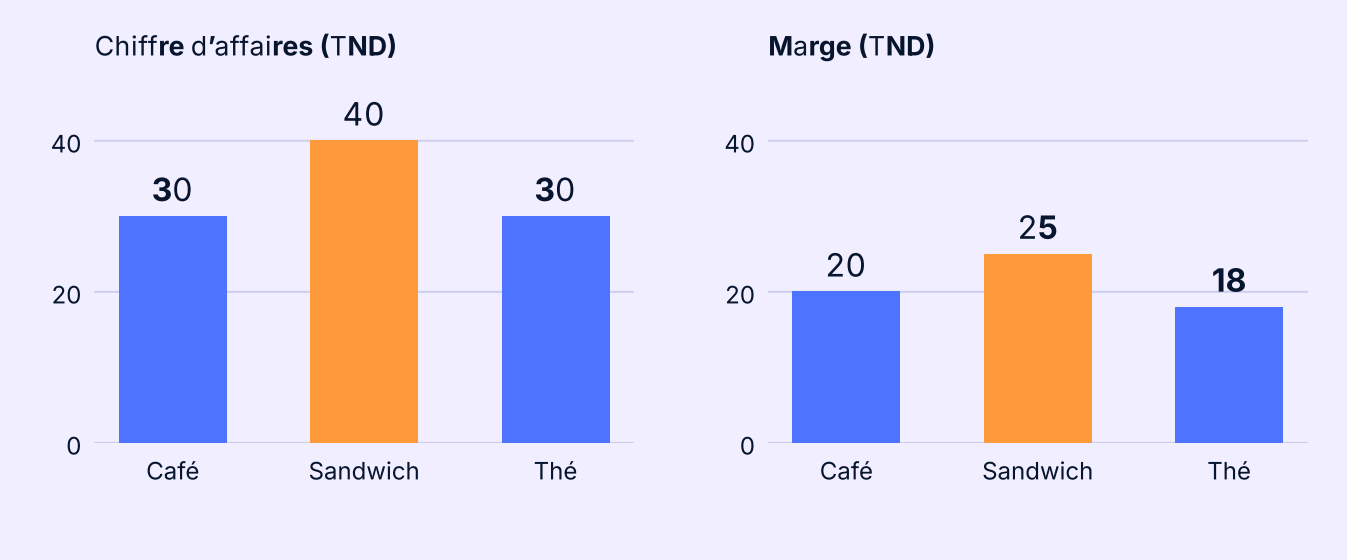
Vérifier le nom du dossier outputs puis relancer png(), barplot() et dev.off() dans cet ordre. Un fichier ouvert par png() n'est pas terminé tant que dev.off() n'a pas été exécuté.

## Deux graphiques, deux questions

| Graphique | Ce que vous devez pouvoir lire |
|---|---|
| Ventes | Cafe 30, Sandwich 40, The 30 TND |
| Marges | Cafe 20, Sandwich 25, The 18 TND |

MISSION 6 · LA PAUSE NORIA

# Deux graphiques, deux questions




Dans votre tête · Mêmes produits, deux mesures monétaires : ventes et marge.

EN RÉSUMÉ · VOUS SAVEZ MAINTENANT

Transformer un résumé en diagramme en barres, nommer ce que les barres mesurent et enregistrer une image. Vous savez comparer la figure au tableau qui l'a produite.

Dire « le sandwich est le plus haut » ne suffit pas. Dire « le sandwich a le chiffre d'affaires le plus élevé sur ces quatre jours : 40 TND » précise la mesure et la période.

Enregistrer missions/06_visualiser.R et vérifier depuis la console : source("checks/06_visualiser.R"). Le contrôle relance votre script ; les fichiers demandés doivent aussi être lisibles dans outputs.

MISSION 7 · LA PAUSE NORIA

# La gérante demande votre avis

Objectif · Calculer la part des ventes du sandwich, choisir une action prudente et écrire une courte note.

Pourquoi ? Les chiffres deviennent utiles quand ils répondent à une question. La gérante veut savoir quel produit mettre en avant pour son prochain petit test commercial.

| À la main | Calcul | Lecture |
|---|---|---|
| Part du sandwich | 40 / 100 × 100 = 40 | 40 % des ventes de la période |
| Marge du sandwich | 40 − 15 = 25 TND | La plus élevée des trois produits |

UNE PART N'EST PAS UN PRIX

Un pourcentage rapporte une partie au total : partie / total × 100. Son unité est %. Ici, 40 % parle du chiffre d'affaires ; cela ne veut pas dire 40 % des clients ni des unités vendues.

## Sélectionner, puis calculer la part

== pose une question d'égalité. bilan_produits$produit == "Sandwich" donne TRUE pour la ligne sandwich. Des crochets autour de cette règle gardent seulement la valeur correspondante.

MISSION 7 · LA PAUSE NORIA

# Sélectionner, puis calculer la part




Dans votre tête · Sur 100 cases, 40 représentent les ventes du sandwich.

```
ventes_sandwich <-
  bilan_produits$chiffre_affaires[
    bilan_produits$produit == "Sandwich"]
ventes_total <- sum(bilan_produits$chiffre_affaires)
part_sandwich <- ventes_sandwich /
  ventes_total * 100
print(part_sandwich)
```

| Objet | Attendu |
|---|---|
| ventes_sandwich | 40 TND |
| ventes_total | 100 TND |
| part_sandwich | 40 % |

MISSION 7 · LA PAUSE NORIA

# Sélectionner, puis calculer la part

> DEUX SIGNES DIFFÉRENTS
>
> <- donne une valeur à un objet. == vérifie une égalité. Pour chercher Sandwich, respecter sa majuscule et les guillemets ; Sandwich sans guillemets serait un nom d'objet.

## Laisser une décision que l'on peut relire

Dans missions/07_decider.R, créer produit_prioritaire <- "Sandwich" : c'est le produit retenu après lecture du bilan. Calculer part_sandwich puis enregistrer outputs/07_bilan_produits.csv avec write.csv(). La préparation fournie recrée le bilan et prévoit le fichier de référence pour commencer cette mission séparément. L'exercice demande de retenir Sandwich ; votre idée d'action peut varier si vous la justifiez.

writeLines() écrit du texte dans un fichier. c() peut aussi réunir plusieurs lignes de texte. Chaque texte va entre guillemets ; la virgule sépare les lignes. Le résultat sera un fichier .txt lisible sans R.

```
note <- c(
  "Sur quatre jours, le sandwich vend 40 TND.",
  "Cela represente 40 % des ventes.",
  "Je propose un petit test de mise en avant.",
  "Le chiffre d affaires n est pas la marge.",
  "La periode est courte : verifier ensuite."
)
writeLines(note, "outputs/07_decision.txt")
```

- Calculer la part du sandwich dans votre script, puis vérifier le résultat.
- Exporter le bilan demandé avec write.csv(..., row.names = FALSE).
- Rédiger votre note : résultat chiffré, action proposée et limite des données. Vous pouvez proposer une autre action si vous l'expliquez.
- Ouvrir le CSV et la note dans outputs pour vérifier ce qui a vraiment été enregistré. La note doit aussi rappeler que chiffre d'affaires et marge sont deux mesures différentes.

MISSION 7 · LA PAUSE NORIA

# Laisser une décision que l'on peut relire

UNE DÉCISION PROPORTIONNÉE

Ces quatre jours permettent de proposer un test. Ils ne prouvent pas qu'une promotion augmentera les ventes, ni que le résultat sera le même le mois prochain.

## Votre avis s'appuie sur des preuves

| Votre note doit répondre à... | Exemple de réponse |
|---|---|
| Quel résultat ? | Sandwich : 40 TND, soit 40 % des ventes |
| Quelle action ? | Tester une mise en avant limitée |
| Quelle limite ? | Seulement quatre jours, données synthétiques |
| Comment vérifier ensuite ? | Comparer les ventes des prochains jours |

EN RÉSUMÉ · VOUS SAVEZ MAINTENANT

Sélectionner une valeur par son nom, calculer un pourcentage, exporter un tableau et écrire une décision courte. Vous savez distinguer un constat et une proposition à tester.

Vous disposez maintenant d'un chemin complet : fichier brut, nettoyage, calculs, graphique, décision. La dernière mission transforme ce chemin en outil pour un nouveau fichier.

Enregistrer missions/07_decider.R et vérifier depuis la console : source("checks/07_decider.R"). Le contrôle relance votre script ; les fichiers demandés doivent aussi être lisibles dans outputs.

MISSION 8 · LA PAUSE NORIA

# Le prochain marché arrive

Objectif · Enfermer une petite suite de commandes dans une fonction, puis la réutiliser sur un autre fichier.

Pourquoi ? La gérante revient avec une nouvelle caisse. Recopier toutes les commandes augmente le risque d'oublier une étape. Une fonction permet de donner un nom à une recette de calcul et de lui fournir un autre ingrédient.

| Petit exemple à la main | Résultat |
|---|---|
| 3 TND vendus – 1 TND d'ingrédients | 2 TND de marge |
| 8 TND vendus – 3 TND d'ingrédients | 5 TND de marge |

> LE DÉFI EST R, PAS UNE NOUVELLE FORMULE
>
> Nous gardons les mêmes calculs. Ce qui change est la façon de les organiser : une entrée, des instructions et une réponse.

## Votre toute première fonction

function() crée une fonction. Les noms entre parenthèses sont ses entrées, appelées arguments. Les accolades { } entourent ses instructions. return() indique ce qu'elle renvoie. <- donne un nom à cette recette.

```
calculer_marge <- function(vente, cout) {
  resultat <- vente - cout
  return(resultat)
}
print(calculer_marge(3, 1))
print(calculer_marge(8, 3))
```

| Appel | Réponse attendue |
|---|---|
| calculer_marge(3, 1) | 2 TND |
| calculer_marge(8, 3) | 5 TND |

MISSION 8 · LA PAUSE NORIA

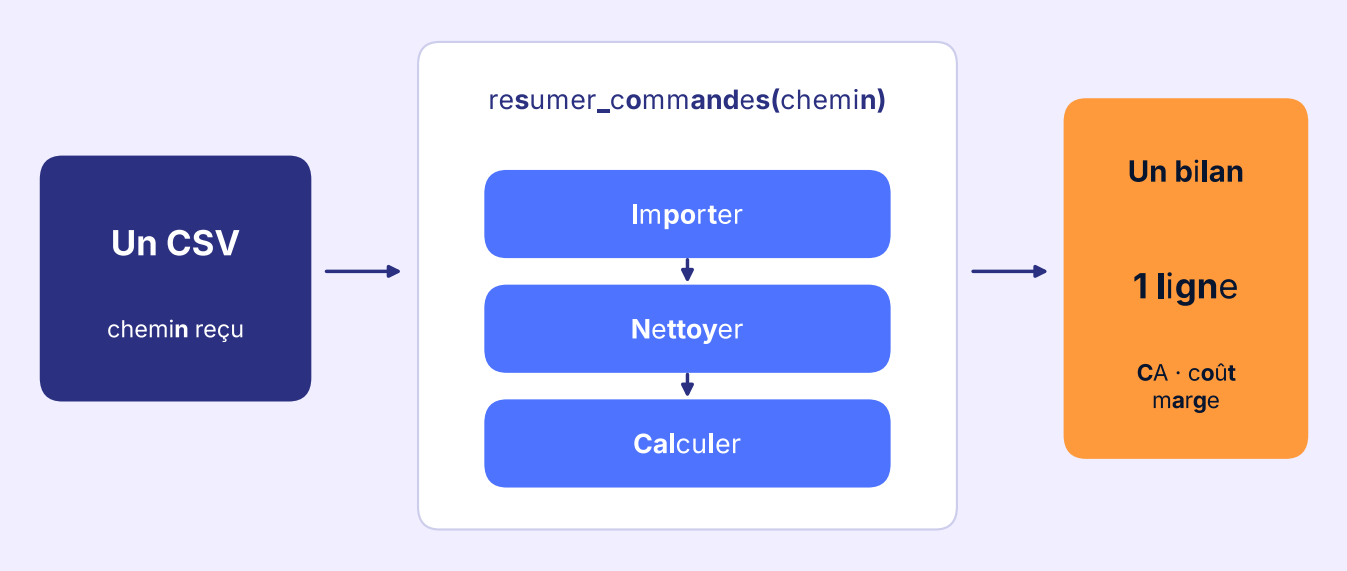
# Votre toute première fonction

DÉFINIR PUIS APPELER

Exécuter seulement la définition ne montre pas de réponse : elle prépare la fonction. Pour l'utiliser, écrire son nom avec des valeurs entre parenthèses. Toujours définir la fonction avant son premier appel.

## Recevoir un fichier, rendre un bilan

La fonction reçoit un chemin CSV. data.frame() construit un petit tableau : chaque nom suivi de = crée une colonne. Ici, chaque colonne contient une seule valeur ; la réponse n'a donc qu'une ligne. return() renvoie ce tableau.




Dans votre tête · Un chemin entre, les mêmes étapes se suivent, un bilan sort.

```
resumer_commandes <- function(chemin) {
  x <- read.csv(chemin)
  x <- unique(x)
  x <- x[!is.na(x$quantite), ]
  ventes <- sum(x$quantite * x$prix_unitaire)
  couts <- sum(x$quantite * x$cout_unitaire)
  return(data.frame(chiffre_affaires = ventes,
    cout_total = couts, marge = ventes - couts,
    quantite = sum(x$quantite),
    n_commandes = nrow(x)))
}
```

MISSION 8 · LA PAUSE NORIA

# Recevoir un fichier, rendre un bilan

x est un nom local de tableau, choisi pour raccourcir les lignes. chemin reçoit le chemin fourni à l'appel. La recette résume ; la conservation séparée des lignes à compléter reste celle de la mission 4.

## La même recette, une autre caisse

Dans missions/08_automatiser.R, définir resumer_commandes() puis l'appeler deux fois. Les deux affectations ci-dessous enregistrent ses réponses dans deux objets différents.




Dans votre tête · La recette reste la même ; les fichiers couvrent des durées différentes.

```
resultat_semaine_1 <- resumer_commandes(
  "data/orders_raw.csv")
resultat_semaine_2 <- resumer_commandes(
  "data/orders_week2_raw.csv")
print(resultat_semaine_1)
print(resultat_semaine_2)
```

Premier carnet : quatre jours. Deuxième carnet : deux jours. Les noms semaine_1 et semaine_2 désignent ces carnets ; ils ne signifient pas deux semaines complètes. Les totaux de périodes différentes ne mesurent pas une hausse ou une baisse de performance.

MISSION 8 · LA PAUSE NORIA

# La même recette, une autre caisse

```
write.csv(resultat_semaine_2,
  "outputs/08_resume_semaine_2.csv",
  row.names = FALSE)
```

VOTRE ESSAI ANALOGUE

Le deuxième fichier contient 8 lignes brutes, dont une copie exacte et une quantité manquante. Votre fonction doit retrouver 6 commandes utilisables et 9 unités, puis 48 − 18 = 30 TND de marge.

## Votre outil sait recommencer

| Nouvelle caisse nettoyée | Attendu |
|---|---|
| Ventes utilisables | 6 lignes |
| Quantité totale | 9 unités |
| Chiffre d'affaires | 48 TND |
| Coût des ingrédients | 18 TND |
| Marge | 30 TND |

EN RÉSUMÉ · VOUS SAVEZ MAINTENANT

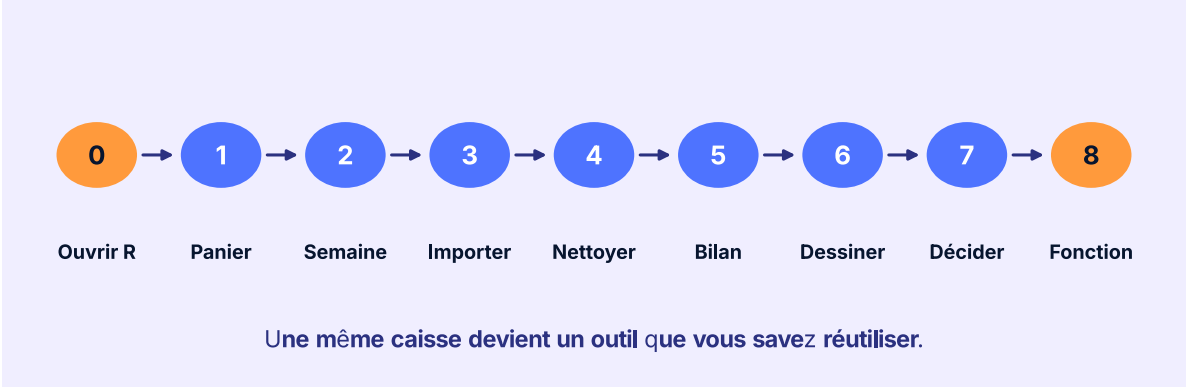
Définir une fonction simple, lui fournir des arguments, lire sa réponse et la réutiliser. Vous savez que la fonction doit être redéfinie en relançant le script après un redémarrage de R.

Vous n'avez pas appris une formule plus difficile. Vous avez appris à organiser une méthode que vous comprenez pour ne pas la réécrire à chaque nouveau fichier.

Enregistrer missions/08_automatiser.R et vérifier depuis la console : source("checks/08_automatiser.R"). Le contrôle relance votre script ; les fichiers demandés doivent aussi être lisibles dans outputs.

FIN · GARDER LA MÉTHODE

# Le café est devenu un outil




Dans votre tête · La même caisse grandit, une mission après l'autre.

| Vous savez... | Vous pouvez le montrer en... |
|---|---|
| Écrire du R | Relançant un script sauvegardé après fermeture. |
| Organiser les valeurs | Créant un vecteur, puis lisant lignes et colonnes. |
| Contrôler les données | Expliquant copie exacte, quantité absente et mise à part. |
| Calculer et regrouper | Retrouvant 100 TND de ventes, 37 de coût et 63 de marge. |
| Communiquer clairement | Présentant deux graphiques et une décision prudente. |
| Réutiliser votre travail | Appliquant votre fonction au nouveau fichier. |

FIN · GARDER LA MÉTHODE

# Le café est devenu un outil

UNE MÉTHODE À GARDER

Prédire un petit résultat → exécuter → comparer → expliquer → enregistrer. Une sortie surprenante donne une question à résoudre. Revenir à quelques lignes et un calcul à la main aide à comprendre.

## Savoir reprendre, même après une erreur

| Si… | Essayer d'abord… |
|---|---|
| Un fichier est introuvable | Vérifier getwd(), le dossier et le chemin. |
| Un objet est introuvable | Relancer sa création et vérifier son nom. |
| R attend avec + | Échap dans RStudio, Ctrl+C dans le Terminal ; vérifier parenthèses et guillemets. |
| Le résultat paraît faux | Revenir au mini-calcul à la main et à ses unités. |
| L'image ne se crée pas | Relancer png() → graphique → dev.off(). |

Avant de quitter : enregistrer, fermer R, rouvrir le dossier et relancer votre dernier script depuis le début. Retrouver le même résultat montre que votre méthode est bien conservée dans le fichier.

VOTRE TRACE DE PROGRESSION

Ajouter deux commentaires à votre script : « Je sais maintenant… » et « Mon prochain essai sera… ». Conserver les fichiers de outputs/. Ils racontent le chemin parcouru, de la première caisse au prochain fichier.

Si besoin, 02_aides.pdf propose un indice à la fois et 04_memo_R.pdf rappelle les gestes. Après une tentative, 03_corriges.pdf permet de comparer votre démarche. Le parcours principal se trouve entièrement dans ce guide.